Spring Data JPA Relationships

Why Spring Data JPA Relationships Matter in Real Projects ?

In real-world backend systems, **data never exists in isolation**.

Almost every production system involves:

- Parent–child data
- Ownership relationships
- Aggregates
- Referential integrity
- Transactional consistency across tables

Spring Data JPA relationships are **not about annotations alone**.

They directly affect:

- SQL schema design
- Foreign key constraints
- Transaction behavior
- Insert/update/delete order
- Performance (joins, fetching, cascades)
- Serialization (JSON recursion issues)
- Data integrity bugs in production

A wrong relationship design can cause:

- Data inconsistency
- Infinite JSON loops
- N+1 query issues
- Orphan records
- Unexpected deletes

What Is a Relationship Between Entities?

In JPA, a **relationship** defines how **one entity references another entity** and how that mapping translates to **database foreign keys and joins**.

At database level:

- One table holds a **foreign key**
- That foreign key references a **primary key** in another table

At JPA level:

- One entity holds a **Java object reference**
- JPA maps that object reference to a **foreign key column**

Key principle

👉 Relationships are defined in **entities**, not in controllers or services.

Types of Relationships in JPA

Spring Data JPA supports four core relationship types:

1. **OneToOne**
2. **OneToMany**
3. **ManyToOne**
4. **ManyToMany**

Each has:

- A database meaning
- An owning side
- A non-owning (mappedBy) side
- Cascade and fetch implications

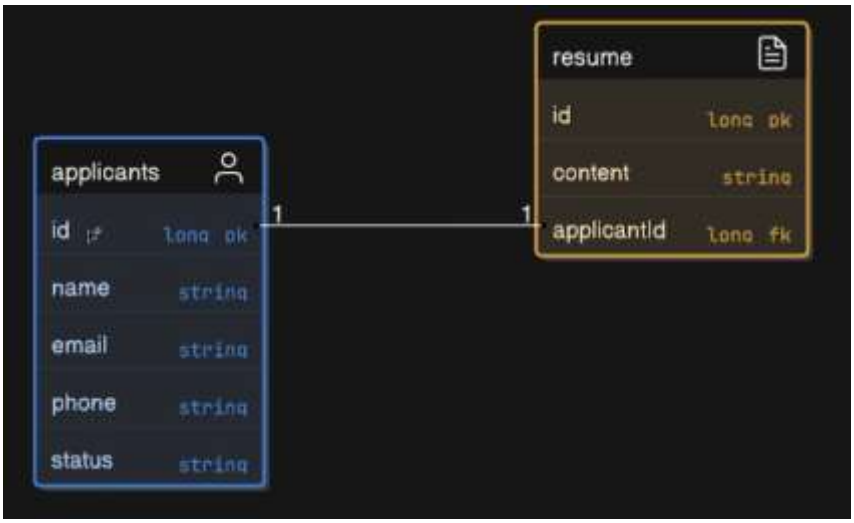
One-to-One Relationship – Deep Dive (AMS Scenario)

Scenario: Application Management System (AMS)

- **Applicant** → can upload **only one Resume**
- **Resume** → belongs to **only one Applicant**

This is a **strict one-to-one relationship**.

Database View



Entity Relationship (ER) Concept

- Resume holds the **foreign key**
- Resume is the **owning side**
- Applicant is the **inverse side**

OneToOne – Entity Implementation (Unidirectional)

Applicant Entity

```
@Entity
public class Applicant {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String email;
    private String phone;
    private String status;
}
```

Resume Entity (Owning Side)

```
@Entity
public class Resume {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String content;

    @OneToOne
    @JoinColumn(name = "applicantId", nullable = false)
    private Applicant applicant;
}
```

What Actually Happens Internally → unidirectional mapping

- @JoinColumn creates **applicantId column**
- JPA enforces **foreign key relationship**
- Resume table holds the FK → Applicant table
- Applicant entity is **not aware** of Resume

Resume API Flow (Unidirectional)

ResumeController

```
@RestController
@RequestMapping("/resumes")
public class ResumeController {

    @Autowired
    private ResumeService resumeService;

    @PostMapping("/{applicantId}/resume")
    public ResponseEntity<Resume> addResume(
            @PathVariable Long applicantId,
            @RequestBody Resume resume) {

        return ResponseEntity.ok(resumeService.addResume(applicantId, resume));
    }
}
```

ResumeService

```
@Service
public class ResumeService {

    @Autowired
    private ApplicantJpaRepository applicantJpaRepository;

    @Autowired
    private ResumeRepository resumeRepository;

    public Resume addResume(Long applicantId, Resume resume) {

        Applicant applicant = applicantJpaRepository.findById(applicantId)
                .orElseThrow(() ->
                        new RuntimeException("Applicant not found with id: " + applicantId));

        resume.setApplicant(applicant);
        return resumeRepository.save(resume);
    }
}
```

ResumeRepository

```
public interface ResumeRepository extends JpaRepository<Resume, Long> {
}
```

Bidirectional OneToOne Mapping

Now we want:

- Resume ↔ Applicant (both directions)

Applicant Entity (Inverse Side)

```
@OneToOne(mappedBy = "applicant")
private Resume resume;
```

Now:

- Resume owns the relationship
- Applicant is the inverse side

Problem: Insert Order Issue

If you try to **save Applicant with Resume directly**, JPA throws an error because:

- Resume needs applicantId
- Applicant is not saved yet
- applicantId does not exist

Solution 1: Manual Save Order

In ApplicantService:

1. Save Applicant
2. Set Applicant in Resume
3. Save Resume

This works but is **error-prone and repetitive**.

Solution 2: Cascading (Correct Approach)

Add Cascade

```
@OneToOne(mappedBy = "applicant", cascade = CascadeType.ALL)
private Resume resume;
```

What CascadeType.ALL Means

- PERSIST → save child when parent saved
- MERGE → update child automatically
- REMOVE → delete child when parent deleted
- REFRESH, DETACH

Cascade controls **entity lifecycle propagation**

New Problem: Circular JSON Serialization

Now API returns:

- Applicant → Resume → Applicant → Resume → ∞

Fix: @JsonIgnore

Resume Entity

```
@OneToOne
@JoinColumn(name = "applicantId", nullable = false)
@JsonIgnore
private Applicant applicant;
```

Why This Works

- Jackson ignores applicant field during serialization
- Breaks infinite recursion
- Does **not** affect database relationship

@JsonIgnore is a **serialization-level fix**, not a JPA fix.

OneToMany & ManyToOne Relationship

Scenario: Applicant → Applications

- One Applicant → Many Applications
- Many Applications → One Applicant

Application Entity (Owning Side)

```
@Entity
public class Application {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String applicationStatus;

    @ManyToOne
    @JoinColumn(name = "applicantId", nullable = false)
    private Applicant applicant;
}
```

Applicant Entity

```
@OneToMany(mappedBy = "applicant", cascade = CascadeType.ALL)
private List<Application> applications = new ArrayList<>();
```

Key Concept

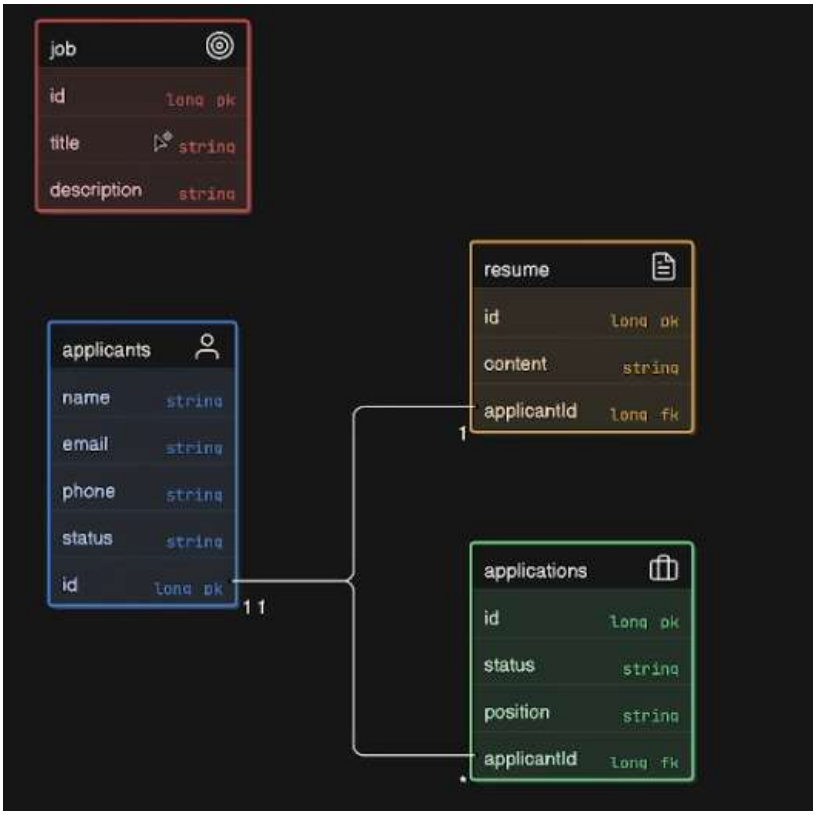
- @ManyToOne is always the **owning side**
- Foreign key exists in **many-side table**
- OneToMany is almost always **mappedBy**

ManyToMany Relationship – Job Scenario

Scenario

- One Applicant → Many Jobs
- One Job → Many Applicants

This creates a **join table**.



Applicant Entity

```
@ManyToMany
@JoinTable(
        name = "applicant_job",
        joinColumns = @JoinColumn(name = "applicantId"),
        inverseJoinColumns = @JoinColumn(name = "jobId")
)
private List<Job> jobs = new ArrayList<>();
```

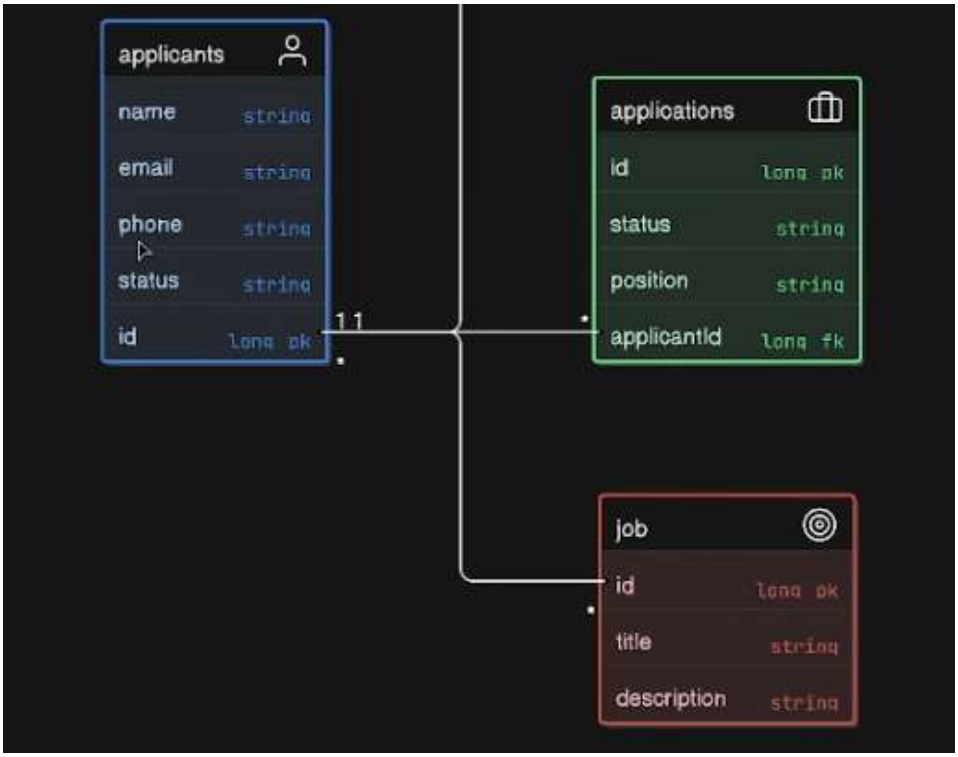
Job Entity

```
@ManyToMany(mappedBy = "jobs")
private List<Applicant> applicants = new ArrayList<>();
```

Database Result

applicant_job

- applicantId
- jobId



Spring Data JPA Relationships – Scenario-Wise Quick Summary

One-to-One Relationship

Scenario: Applicant ↔ Resume

Meaning

- One Applicant → One Resume
- Resume holds the foreign key (applicantId)

Owning Side

- Resume entity

Annotations Used

- @OneToOne
- @JoinColumn(name = "applicantId")
- mappedBy (in Applicant)
- CascadeType.ALL
- @JsonIgnore

Key Points

- Foreign key exists in Resume table
- mappedBy avoids duplicate FK
- Cascade handles save/delete order
- @JsonIgnore prevents infinite JSON loop

One-to-Many / Many-to-One Relationship

Scenario: Applicant → Applications

Meaning

- One Applicant → Many Applications
- Many Applications → One Applicant

Owning Side

- @ManyToOne (Application entity)

Annotations Used

- @ManyToOne
- @JoinColumn(name = "applicantId")
- @OneToMany(mappedBy = "applicant")
- CascadeType.ALL

Key Points

- Foreign key always on many-side
- @ManyToOne controls DB updates
- @OneToMany is inverse side
- Parent must be set in child before save

Many-to-Many Relationship

Scenario: Applicant ↔ Job

Meaning

- One Applicant → Many Jobs
- One Job → Many Applicants

applicantId	jobId
1	101
1	102
2	101
3	103

Database

- Join table (applicant_job)

Annotations Used

- @ManyToMany
- @JoinTable
- mappedBy

Key Points

- Join table holds both FKs
- Avoid in large systems
- Prefer join entity for scalability

Cascade – Must Know

Purpose

- Controls entity lifecycle propagation

Common Usage

- CascadeType.ALL → Save/Delete child with parent

Important

- Cascade ≠ Relationship
- Cascade ≠ Join

Serialization Handling

Problem

- Bidirectional mapping causes infinite loop

Solution

- @JsonIgnore on back reference

Note

- Affects JSON only, not DB behavior

Must-Remember Rules

- Owning side controls foreign key
- mappedBy = inverse side
- Many-side usually owns relationship
- Cascade manages lifecycle, not joins
- Always control JSON serialization